Course Outline

Logout

Course Outline

Contents

1. Course Details
2. Course Summary
3. Philosophy of teaching web-based front-ends
4. Teaching Strategies
5. Assessment
6. Schedule / Timetable
7. Student Conduct
8. Resources for Students
9. Course Evaluation and Development

1. Course Details

Find information relating COVID-19 and this offering here and here.

Course Code	COMP6080
Course Title	Web Front-end Programming
Convenor	Hayden Smith
Admin	Hayden Smith

Course Outline

Units of Credit	6
Course Website	This website!
Handbook Entry	http://www.handbook.unsw.edu.au/undergraduate/courses/current/COMP6080.html

---

2. Course Summary

COMP6080 aims to develop your confidence in building modern web-based applications to industry standards. This occurs through the introduction of a range of basic concepts surrounding HTML, CSS, Vanilla Javascript, Javascript Declarative Frameworks, UI/UX Principles, Accessibility, Network & Asynchronous Programming, Front-end Testing, and other basic infrastructure.

This course has a heavy emphasis on industry voices, and as such a number of lectures will be given by current front-end developers from industry. These lectures primarily come from employees at Canva, a Sydney-based technology company that does a lot of work with front-end technologies.

COMP6080 is a challenging course. Front-end development is unlike most things you've experienced at university before. You will find the individual problems you solve much simpler than other level 6 courses, but the time you will feel that you spend on the aggregate of these issues will feel larger. A number of students will find this course quite time consuming if they're hoping to achieve a high mark. We'd encourage you to reflect on this fact before you enrol in the course.

2.1. Assumed Knowledge

Course Outline

COMP6080 assumes that you have satisfactorily completed COMP1531 and COMP2521. The main areas of knowledge you'll need from these courses are:

- A high level understanding of how interpreted scripting languages work (e.g. python), in terms of inputs, interpretation, loose typing, etc
- Confident use of GIT as a version management tool
- Basics of HTTP protocol and interacting with web browsers

Postgraduate students will need to familiarise themselves with git, specifically, if not already confident, check out the resources here. We have also provided a git-intro exercise that we encourage everyone who hasn't completed COMP1531 to complete prior to the beginning of the course. Basic competency in git is an expectation in COMP6080.

2.2. Course Learning Outcomes

After completing this course, students will be able to:

1. CLO1 : Able to apply Javascript semantics to design, construct, test and debug programs holistically
2. CLO2 : Construct programs for web-front end with HTML, CSS, and DOM manipulation
3. CLO3 : Use Javascript and CSS frameworks to allow more efficient integration of existing code and components into a final product
4. CLO4 : Build stable applications that utilise concurrent programming through use of Javascript's asynchronous programming techniques
5. CLO5 : Design and build interfaces that focus on best user experience and accessible design practices

Course Outline

3. Philosophy of teaching web-based front-end

3.1. Challenges of teaching front-end

The challenges of teaching front-end in a finite time period are related to the size and scope of knowledge around modern web front-end. Building even the most basic modern web app requires an extreme breadth and depth of abstractions that take typical developers years to become very comfortable with.

Teaching front-end is also challenging due to the rapid advances that are made in the world of web (a good thing), which leads to sprawling and conflicting resources across the internet. This can lead to learning inconsistent or sub-standard practices and programming patterns.

To teach only declarative frameworks (e.g. ReactJS) as an introduction creates the problem of producing capable students who can rapidly produce work, but do so without the fundamental understanding to grow and apply their skills sustainably and with accountability. However, to teach only Vanilla Javascript would neglect the reality that nearly all students in this course will inevitab not be building actual applications with Vanilla Javascript.

3.2. Approach to teaching

This course is about spending a term learning both Vanilla Javascript (Javascript without abstractions) and Javascript Declarative Frameworks (specifically ReactJS). The term will begin with a heavy focus on HTML/CSS, and have topics of UI/UX, Accessibility, and Testing, trickled throughout. The course will be very heavy Vanilla JS to start while slowly tapering off, and will be very light on ReactJS at the start before rapidly getting into more depth.

Course Outline

3.3. What isn't included

There will be a number of people who feel strongly about the exclusion of particular topics from the course, such as typescript or more complex state managers (redux, mobx). Often when topics are omitted it's because they aren't *fundamental* knowledge in the limited weeks this course is offered in.

We've compiled an FAQ to answer these questions. If you still have further questions or comments, we'd encourage you to use the forum linked in the sidebar.

Students with prerequisite knowledge in this course should understand that this is an introductory course with very limited assumed knowledge, and as such, is not a course to extent an already firm foundation of knowledge. If you are already quite competent in the areas taught in this course, please be patient as we will be moving quite slowly by your standards. If this is something that you feel may be frustrating, it may be more appropriate with your skill-set to find a more stimulating course.

---

4. Teaching Strategies

This course uses the standard set of practice-focused teaching strategies employed by most CSE foundational courses:

- Lectures
- Tutorials
- Exercises
- Help Sessions
- Assignments

Course Outline

- Final Exam

4.1. Lectures

Lectures will be used to present the theory and practice of the techniques in this course. Although the lectures will primarily focus on the key concepts of software engineering, some lectures will also include practical demonstrations. Lecture slides will be available on the course web page.

On a typical week we will only have the live lecture (2 hours). The live lecture will be replaced by approximately 2 hours of pre-recorded lectures to watch each week. Please note:

- It's critical that you watch the 2 hours of pre-recorded lectures each week *prior* to the live lecture
- The length of pre-recorded and live lectures may vary week to week, but the intention is to keep the average weekly duration at 4 hours across the term.
- In general, industry driven lectures are more likely to be in the pre-recorded format
- Lecture notes for the live lectures may only be released a day or two before the lecture. This because as part of the teaching strategy for this course the live lectures are designed to essentially "supplement" and "round-off" the pre-recorded lectures, so it's natural to wait for some feedback from students before over-planning these.

You can see the schedule for lectures here, and the list of lecture videos and content here.

Because lectures will be delivered from a *range* of people, it's important that students are prepared for differences in slide format and teaching style.

4.2 Tutorials

Course Outline

Tutorials are 1 hour lessons every week that help clarify ideas from lectures and work through exercises based on the lecture material. You should make sure that you use them effectively by examining in advance the material to be covered in each week's tutorial, by asking questions, by offering suggestions and by generally participating.

Tutorial activities can be found by going to the tutorials page. Tutorial solutions are available on the solution branch of the exercises gitlab repo. Tutorials do not contribute to your final mark.

Tutorials will be run mostly in person, but sometimes over MS teams.

4.3. Help Sessions

Help sessions are unprepared drop-in "clinics" where students and groups can go to seek help about course related matters, whether that be the project, tutorials, or exercises. Current tutors or lab assistants will supervise each help session.

Help sessions will be run via MS teams.

A timetable for help sessions can be found here.

4.4. Assignments

There will be a series of assignments which will run through the teaching period from weeks 1-10 and contributes to **80%** of the overall course mark.

Assignments will be the platform you have to study and learn the material in substantially more depth. Your assignments will be completed via gitlab.

The intention is that the third assignment will be completed in a pair. You are able to opt out of this and work alone (at your own expense!). You can choose your pair, otherwise we will assign you a partner. You can pair up with any student in the course.

Course Outline

Information on assignments can be found in the links in section 5 (Assessment).

4.6. Final Exam

There will be a centrally timetabled final exam which will in your UNSW exam timetable. The exam may contain a mixture of multiple choice questions, short answer questions, and programming exercises. More specific details of the exam will be provided through the course.

If you cannot attend the final exam because of illness or misadventure, then you must submit a Special Consideration request, with documentation, through MyUNSW within 72 hours of the start of the exam exam. If your request is reasonable, then you will be awarded a Supplementary Exam. No supplementary exams will be provided for students who score marks 49 or below on grounds of being "close" to a pass.

Information on the final exam can be found in the link in the sidebar.

5. Assessment

Assess	Weighting	Due Date	Individual/Pair	Link
Ass 1	15%	Week 3, Monday	Individual	View
Ass 2	5%	Week 4, Monday	Individual	View
Ass 3	30%	Week 7, Monday	Individual	View

Course Outline

Assess	Weighting	Due Date	Individual/Pair	Link
Ass 4	30%	Week 10, Monday	Pair (see below)	View
Exam	20%	Exam Period	Individual	View

For pair assignments, you complete them a pair of your choice. If you don't have a pair, we will find one for you. You can also choose to work alone (this is something we arguably should not offer, but we do because we know there are exceptional circumstances), but we strongly do not recommend this as the workload is much higher for an individual. If the workload is a concern, work as a pair (how it's designed).

Nominations for your pair (or to work alone) must be complete by the end of week 5. Information about this will be distributed in weekly notices.

Pairs will be **required** to contribute regularly to gitlab and in reasonably equal contributions as we still assess contributions individually (there is no blanket group mark assigned). Failure to do so may result in a loss of marks.

---

6. Course Schedule / Timetable

The timetable for this course is outlined clearly in the timetable for lectures, tutorials, and help sessions.

The content schedule is outlined below:

Course Outline

Week 1

- Course Overview
- Intro to Git
- HTML Fundamentals
- Image Types
- CSS Rules
- More Tags
- CSS Formatting
- CSS Layouts
- Flexbox
- SVGs
- Pre-processors
- zindex
- CSS Showcase
- Fonts

Week 2

- Demo: HTML/CSS Page
- Mobile CSS
- Using CSS Frameworks
- CSS Grids

Course Outline

- Dev Tools
- Javascript Intro (compared to C)
- Javascript Language Features & Syntax
- Javascript Advanced Functions
- The Javascript Ecosystem
- Node Package Manager
- NPM Advanced

Week 3

- Demo: Javascript - NodeJS
- Introduction
- DOM
- Events
- Javascript Closures
- Forms
- Local Storage

Week 4

- Demo: Javascript - Browser
- Events & Callbacks
- Promises
- Await / Async

Course Outline

- AJAX
- XMLHttpRequest (XHR)
- Fetch
- JSON & Data-interchange formats
- Understand HTTP Servers

Week 5

- Demo: Ass2 - Async & Planning
- UI Fundamentals
- Good & Bad UI 1
- Good & Bad UI 2

Week 7

- Using Git in pairs
- Introduction
- Basic "Global CSS" Usage
- Lifescycle
- Transpilation
- useState hook
- Demo: ReactJS Intro & A11y
- useEffect hook
- Using Fetch & Hooks

- Working with Multiple Files
- Components & Props
- Before you code

Week 8

- Demo: ReactJS in Ass3
- Linting
- Routing & SPAs
- CSS Frameworks
- useContext
- State management
- Usability testing
- Figma & Component Libraries
- Pre-rendering

Week 9

- Demo: ReactJS Further
- Perceivability
- Operabilitity
- Understandability
- Robustness
- UI Testing

Course Outline

- UI Testing (Demo)

Week 10

- Final Exam Overview

---

7. Student Conduct

The **Student Code of Conduct** (Information, Policy) sets out what the University expects from students as members of the UNSW community. As well as the learning, teaching and research environment, the University aims to provide an environment that enables students to achieve their full potential and to provide an experience consistent with the University's values and guiding principles. A condition of enrolment is that students *inform themselves* of the University's rules and policies affecting them, and conduct themselves accordingly.

In particular, students have the responsibility to observe standards of equity and respect in dealing with every member of the University community. This applies to all activities on UNSW premises and all external activities related to study and research. This includes behaviour in person as well as behaviour on social media, for example Facebook groups set up for the purpose of discussing UNSW courses or course work. Behaviour that is considered in breach of the Student Code Policy as discriminatory, sexually inappropriate, bullying, harassing, invading another's privacy or causing any person to fear for their personal safety is serious misconduct and can lead to severe penalties, including suspension or exclusion from UNSW.

If you have any concerns, you may raise them with your lecturer, or approach the School Ethics Officer, Grievance Officer, or one of the student representatives.

Plagiarism is defined as using the words or ideas of others and presenting them as your own. UNSW and CSE treat plagiarism as academic misconduct, which means that it carries penalties as

Course Outline

severe as being excluded from further study at UNSW. There are several on-line sources to help you understand what plagiarism is and how it is dealt with at UNSW:

- Plagiarism and Academic Integrity
- UNSW Plagiarism Procedure

Make sure that you read and understand these. Ignorance is not accepted as an excuse for plagiarism. In particular, you are also responsible that your assignment files are not accessible by anyone but you by setting the correct permissions in your CSE directory and code repository, if using. Note also that plagiarism includes paying or asking another person to do a piece of work for you and then submitting it as your own work.

UNSW has an ongoing commitment to fostering a culture of learning informed by academic integrity. All UNSW staff and students have a responsibility to adhere to this principle of academic integrity. Plagiarism undermines academic integrity and is not tolerated at UNSW. Plagiarism at UNSW is defined as using the words or ideas of others and passing them off as your own.

If you haven't done so yet, please take the time to read the full text of

- UNSW's policy regarding academic honesty and plagiarism

The pages below describe the policies and procedures in more detail:

- Student Code Policy
- Student Misconduct Procedure
- Plagiarism Policy Statement
- Plagiarism Procedure

Course Outline

You should also read the following page which describes your rights and responsibilities in the CSE context:

- Essential Advice for CSE Students

---

8. Resources for Students

There is no single text book that covers all of the material in this course at the right level of detail and using the same technology base as we are. The lectures should provide sufficient detail to introduce topics, and you will then study them in further depth in the tutorials, exercises and assignments. For some lectures, further reading material may be given for students who wish to gain a deeper understanding.

---

9. Course Evaluation and Development

This course is evaluated each session using the MyExperience system.

This is being addressed during 26T1.

- Addition of automarking in third assignment
- Improved marking feedback and consistency